Linux Filesystem

03/10/2026 Manas Viral Doshi

1. Introduction to Linux Filesystem:

Think of a filesystem as a digital filing cabinet. Without it, your computer wouldn't know where one piece of data ends and another begins. If you are used to Windows, you are familiar with separate drive letters like C:, D:, or E:. Linux does things differently: everything lives inside a single unified tree. There are no drive letters. Whether it is your main hard drive, a USB flash drive, or a network storage folder, everything attaches to a single starting point called the Root Directory (/).

2. Key Features of Linux Filesystem:

What makes the Linux filesystem distinct and efficient? Here are its core characteristics:

- Single Inverted Tree Structure: All directories and files branch out from a single top-level directory (/).
- "Everything is a File": In Linux, documents, images, hardware devices (like your hard disk, keyboard, or printer), and running processes are all represented as files.
- Case Sensitivity: Filenames distinguish between capital and lowercase letters. Report.txt, report.txt, and REPORT.TXT are three completely separate files in the same folder.
- Built-in Security & Permissions: Every file belongs to a specific owner and group. Access rights (Read, Write, and Execute) are strictly enforced, keeping system core files safe from unauthorized changes.
- Standardized Layout (FHS): Linux follows the Filesystem Hierarchy Standard (FHS), ensuring folders have predictable names and uses across different Linux distributions (like Ubuntu, Fedora, or Debian).

3. Directory Structure

Because Linux organizes everything under /, standard folders exist for specific purposes. Here is a quick guide to the most important folders:

Directory	**Purpose**	**Everyday Analogy**
/ (Root)	The base of the entire system. Every file and folder starts here.	The main tree trunk

/bin & /sbin	Stores essential system commands and utilities (like ls, cp, reboot).	The system's main toolbox
/home	Personal folders for regular users (e.g., /home/alex). Stores desktop, downloads, and documents.	Individual user bedrooms
/root	The home directory for the System Administrator (Superuser). Regular users cannot enter.	The master office keycard room
/etc	System-wide configuration files that control how programs and settings behave.	The control panel / manual
/var	Stores "variable" data that changes constantly, like system logs, mail, and web server databases.	The activity logbook
/tmp	Holds temporary files created by programs. Usually wiped clean when the system restarts.	A scratchpad
/dev	Contains special device files that represent hardware (hard drives, webcams, sound cards).	Hardware plugs
/media & /mnt	Locations where external storage (USB drives, external hard disks) are attached when plugged in.	Guest parking spots

4. Basic Linux Commands

To navigate and manage files in Linux, you type simple commands into the terminal. Here are the core commands explained simply:

Navigation & Viewing

1. `pwd` (Print Working Directory)
    - What it does: Shows your current location in the filesystem tree.
    - Example output: `/home/alex/Documents`
2. `ls` (List)
    - What it does: Displays all files and folders inside your current directory.
    - Useful tip: `ls -l` shows extra details like file size, owner, and modified date.
3. `cd` (Change Directory)
    - What it does: Moves you into a different folder.
    - Examples: `cd Downloads` moves into the Downloads folder; `cd ..` moves up one folder level.

File & Directory Management

4. `mkdir` (Make Directory)
    - What it does: Creates a new empty folder.
    - Example: `mkdir Projects`
5. `touch`
    - What it does: Creates a brand-new empty file instantly.
    - Example: `touch notes.txt`
6. `cp` (Copy)
    - What it does: Duplicates a file or folder to another location.
    - Example: `cp notes.txt /home/alex/Backup/`
7. `mv` (Move / Rename)
    - What it does: Relocates a file to a new folder, or renames it in place.
    - Example: `mv old_name.txt new_name.txt`
8. `rm` (Remove)
    - What it does: Deletes a file.
    - Note: Linux terminal does not have a "Recycle Bin" by default so once it is deleted with `rm`, it is gone.

5. Review Questions:

Version Control:

1. What is Version Control?

Think of Version Control as a "time machine" or a smart, automatic "undo button" for project files. Without version control, people often end up saving multiple clutter files like Project_v1.docx, Project_v2_final.docx, and Project_FINAL_really_final.docx. A Version Control System (VCS) eliminates this mess by tracking all changes within a single project history. It records who made a change, what was changed, and why, allowing you to inspect or restore any past version at any time.

2. Why is Version Control Important?

In modern software development and project management, version control provides four core benefits:

- Safety Net & Instant Undo: If a new update breaks your project, you can instantly revert back to the last known working state.
- Seamless Collaboration: Multiple team members can work on the exact same codebase at the same time without accidentally overwriting each other's work.
- Safe Experimentation (Branching): Developers can create isolated "sandbox" branches to test new features safely without risking the main, production-ready project.
- Clear History & Accountability: Every update comes tagged with a timestamp, author name, and description message, making it easy to trace how a system evolved or find where a bug was introduced.

3. Types of Version Control Systems

Version control systems fall into three main architectures:

System Type	How It Works	Key Pros & Cons	Popular Examples
Local VCS	Stores file history in a simple database on your own local computer.	+ Simple setup. - No team collaboration; if your	RCS (Revision Control System)

		hard drive crashes, all history is lost.	
Centralized VCS (CVCS)	A single central server holds the master copy. Developers "check out" files, edit them locally, and send them back.	+ Easy central management & permissions. - Single point of failure (if the server goes offline, nobody can commit or collaborate).	Subversion (SVN), Perforce
Distributed VCS (DVCS)	Every developer downloads a complete copy of the entire project history onto their own machine.	+ Works completely offline; full redundancy (every PC is a complete backup). - Slightly steeper initial learning curve.	Git, Mercurial

4. Basic Git Commands Cheatsheet:

Git is the world's most widely used Distributed Version Control System. Here is a quick-reference cheatsheet grouped by task:

Setup & Initialization

- git config --global user.name "Your Name"
    - What it does: Sets your identity so your name is attached to every change you save.
- git init
    - What it does: Turns the current folder into a new Git project repository ("repo").
- git clone <repository-url>
    - What it does: Downloads an existing project from a remote server (like GitHub) to your computer.

Tracking & Saving Changes

- git status
    - What it does: Shows current workspace status—which files are modified, staged for saving, or ignored.
- git add <filename> (or git add . to select all files)
    - What it does: Moves changed files to the Staging Area (prepares them to be committed).
- git commit -m "Descriptive message"
    - What it does: Saves a permanent snapshot of staged files with a brief explanatory note.
- git diff
    - What it does: Displays line-by-line differences between your current working files and the last saved version.
- git log
    - What it does: Shows the chronological history of all saved commits in the repository.

Branching & Merging

- git branch
    - What it does: Lists all local branches in your project.
- git checkout -b <branch-name> (or git switch -c <branch-name>)
    - What it does: Creates a new working branch and switches you over to it instantly.
- git merge <branch-name>
    - What it does: Combines changes from specified branch into your current active branch.

Cloud & Team Syncing (Remote Repos)

- git fetch
    - What it does: Downloads remote updates from a cloud server without altering your local working files yet.
- git pull
    - What it does: Downloads updates from the remote server and merges them directly into your active files.
- git push
    - What it does: Uploads your locally saved commits to the remote repository (e.g., GitHub, GitLab).

Undoing & Recovery

- git restore <filename>
    - What it does: Discards unsaved local edits in a file, restoring it back to the last commit.
- git stash
    - What it does: Temporarily shelves your uncommitted changes so you can work on a clean slate.
- git stash pop
    - What it does: Brings back your temporarily shelved changes.
- git reset --hard <commit-id>
    - What it does: Rewinds your project to a specific past commit, erasing all edits made after it.

Q) Review Questions for Report 2

1. Git Reset vs. Git Revert

- Git Reset (The "Time Machine / Ctrl+Z"): This command literally rewinds history. It moves your project back to a previous commit as if the newer ones never happened.
    - When to use: Use it when working locally on your own private branch and you want to clean up or undo a mess before anyone else sees it.
    - Catch: Never use this on shared main branches. If you erase commits that your teammates already pulled, you will break their repository.
- Git Revert (The "Counter-Commit"): Instead of deleting history, this creates a new commit that applies the exact opposite changes of a previous commit. If commit #A added a line, git revert #A creates commit #B that deletes that line.
    - When to use: Use it for shared/public branches. It keeps the commit history intact so everyone can see what went wrong and how it was fixed without breaking anyone’s work.

2. Git Fetch vs. Git Pull

- Git Fetch (The "Look Before You Leap"): This checks the remote server (like GitHub) and downloads all the latest changes and commits your teammates made, but it does not touch your actual code files. It just updates your local record of what's happening on the server.
    - Analogy: It’s like checking your mailbox to see what letters arrived, but leaving them sitting on your counter unopened.
- Git Pull (The "Automatic Update"): This downloads the latest changes from the server and immediately merges them directly into your working code files.
    - Formula: git pull is literally git fetch + git merge in one single step.
    - Analogy: It’s like opening the mail and instantly acting on whatever instructions were inside.

3. How Many Commands are in git config?

Technically, git config is a single base command, but it accepts dozens of sub-options, flags, and hundreds of configuration variables.

Rather than memorizing a number, think of it in 3 main scopes/levels:

1. git config --system: Settings for every user on the entire computer.
2. git config --global: Settings for your user profile across all your Git projects (like setting your overall username and email).
3. git config --local: Settings specific to just the current project you are sitting in (e.g., using a work email for a company project and a personal email for side projects).

Rovers:

Q) 6-DOF vs. 9-DOF IMU: What's the Real Difference?

An IMU (Inertial Measurement Unit) is essentially a rover's inner ear—it tells the rover how it's moving, tilting, and balancing.

- 6-DOF (Degrees of Freedom) IMU: Measures 6 directions of movement using two core sensors:
    - 3-Axis Accelerometer: Detects linear motion (forward/backward, left/right, up/down).
    - 3-Axis Gyroscope: Detects rotational motion (pitch, roll, and yaw).
- The Catch: While a 6-DOF IMU knows if the rover is tilting or spinning, it slowly drifts over time when measuring heading (yaw). Without a fixed reference point like a compass, it forgets which way is true North.
- 9-DOF IMU: Takes a standard 6-DOF IMU and adds a third sensor:
    - 3-Axis Magnetometer (Digital Compass): Measures magnetic fields in 3D space.
- Why it matters: The magnetometer gives the 9-DOF IMU an absolute reference point (magnetic North). This cancels out heading drift, making 9-DOF far superior for outdoor navigation across large distances.

Q) What are the 3 Degrees of Freedom for the Magnetic Component?

The magnetometer inside a 9-DOF IMU measures the strength and direction of the Earth's magnetic field along three spatial axes:

1. X-Axis (Forward/Backward Field Strength): Measures magnetic intensity along the rover's longitudinal axis.

2. Y-Axis (Left/Right Field Strength): Measures magnetic intensity along the rover's lateral axis.
3. Z-Axis (Up/Down Field Strength): Measures magnetic intensity pointing straight into or out of the ground.

By combining all three measurements, the rover calculates its exact 3D tilt-compensated orientation relative to magnetic North.

Q) Encoder vs. IMU: Which is Better for a Robotic Arm?

If you are designing a robotic arm, Encoders win hands down. Here is why:

- Encoders (Winner for Robotic Arms): Encoders are attached directly to the motor shafts or joints. They measure the exact angular position of each joint relative to the arm's base. They don't care about gravity or sudden jolts—they simply count motor rotations to tell you precisely where the arm hand is positioned.
- IMUs (Not ideal for Arm Joints): IMUs measure acceleration and gravity. If the arm moves quickly, stops abruptly, or experiences vibration from a drill, the IMU gets confused by the false acceleration forces. Plus, IMU drift accumulates over time, causing positional errors.

  Rule of Thumb: Use Encoders for joint positions and precise arm kinematics. Use an IMU on the end-effector (gripper) only if you need to detect external vibrations, impacts, or terrain slants.

Q) Why is a Stereo Camera Digital?

A Stereo Camera works just like human eyes—it uses two lenses spaced slightly apart to calculate depth and build 3D point clouds of the terrain.

It must be digital because depth perception requires heavy computer math:

1. Pixel-by-Pixel Matching: The rover's computer must compare individual pixels from the left frame to the right frame to calculate distance (disparity).
2. Real-time Processing: Digital image sensors output matrix arrays of numbers directly into the onboard computer (like an Nvidia Jetson or Raspberry Pi) for immediate processing.

An analog camera outputs a continuous, unstructured electrical signal (like old TV video). To do depth mapping with an analog signal, you would have to convert every frame to digital numbers anyway—making pure digital stereo vision mandatory.

Q) Analog vs. Digital Cameras: Where to Use Which?

Digital Cameras (Best for Autonomous & High-Precision Tasks)

Digital cameras break images down into clean grids of numerical data (pixels).

- Where to use them:
    - Autonomous navigation, obstacle detection, and stereo depth estimation.
    - Science inspection tasks where crisp details and raw pixel processing are required.
- Why: The onboard computer can directly analyze digital pixel values without image quality degrading over distance.

Analog Cameras (Best for Low-Latency Manual Teleoperation)

Analog cameras send video as a continuous radio frequency wave (like old FPV racing drones).

- Where to use them:
    - Real-time manual driving via video link (FPV teleoperation).
    - Weight-sensitive or power-constrained micro-drones/scouts attached to the rover.
- Why: Analog video has near-zero latency (lag). When digital signals drop in quality, the video freezes or buffers; when analog signals degrade, you get static, but you can still see where you are driving without losing control.